

**IMPLEMENTASI DAN ANALISIS KOMPLEKSITAS
ALGORITMA BRANCH AND BOUND DAN BACKTRACKING
PADA STUDI KASUS OPTIMASI PEMILIHAN BARANG
MENGUNAKAN 0/1 KNAPSACK PROBLEM**



Disusun Oleh:

Kelompok 11

Muhammad Dimas Setiaji	123240240
Bintang Shada Kawibya Putra	123240247
Ajrun Ramadhan	123240254
Noven Bugijangge	123240264

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2025/2026**

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas segala rahmat dan karunia-Nya sehingga laporan yang berjudul “Implementasi dan Analisis Kompleksitas Algoritma Branch and Bound dan Backtracking pada Studi Kasus Optimasi Pemilihan Barang Menggunakan 0/1 Knapsack Problem” dapat diselesaikan dengan baik dan tepat waktu.

Laporan ini disusun untuk memenuhi tugas mata kuliah Analisis Algoritma. Dalam laporan ini dibahas penerapan dua algoritma, yaitu Branch and Bound dan Backtracking, untuk menyelesaikan permasalahan optimasi pemilihan barang pada 0/1 Knapsack Problem. Selain implementasi program menggunakan bahasa C++, laporan ini juga memuat analisis kompleksitas serta perbandingan kinerja kedua algoritma dalam memperoleh solusi optimal.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah yang telah memberikan arahan dan materi pembelajaran, serta kepada seluruh anggota kelompok yang telah berkontribusi dalam proses penyelesaian tugas ini.

Penulis menyadari bahwa laporan ini masih memiliki berbagai kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa mendatang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat serta menambah wawasan pembaca mengenai penerapan algoritma Branch and Bound dan Backtracking dalam menyelesaikan permasalahan optimasi.

Yogyakarta, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I.....	5
PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Rumusan Masalah	5
Algoritma mana yang lebih efisien untuk studi kasus ini? Error! Bookmark not defined.	
BAB II	6
ANALISIS DAN LANGKAH PENYELESAIAN	6
2.1 Deskripsi Studi Kasus	6
2.2 Konsep Dasar Branch and Bound	6
2.3 Rumus Bound.....	7
BAB III LANGKAH PENYELESAIAN SECARA MANUAL	8
3.1 Contoh Data	8
3.2 Node Awal (Root)	8
3.3 Percabangan Barang A	8
3.4 Percabangan Setelah Mengambil A	8
3.5 Percabangan Setelah Mengambil A dan Tidak Mengambil B	8
3.6 Percabangan Setelah Mengambil A dan C	9
3.7 Hasil Akhir Branch and Bound	9
BAB IV	10
METODE BACKTRACKING	10
4.1 Cara Kerja Backtracking	10
4.2 Pohon Pencarian Backtracking	10
4.3 Keunggulan dan Keterbatasan Backtracking	11
BAB V.....	12
KOMPLEKSITAS ALGORITMA.....	12
5.1 Kompleksitas Backtracking	12
5.2 Kompleksitas Branch and Bound.....	12
5.3 Tabel Perbandingan.....	13
BAB VI	15

PENJELASAN IMPLEMENTASI PROGRAM.....	15
6.1 Source Code	15
6.1.1 File branch_and_bound.cpp	15
6.1.2 File backtracking.cpp	25
6.2 Struktur Data yang Digunakan.....	32
6.3 Implementasi Branch and Bound	32
6.4 Implementasi Backtracking.....	33
BAB VII	34
KESIMPULAN.....	34
KONTRIBUSI ANGGOTA KELOMPOK	36

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia optimasi, salah satu permasalahan yang sering dijumpai adalah bagaimana memilih sejumlah barang dengan batasan kapasitas tertentu agar keuntungan yang diperoleh menjadi maksimal. Permasalahan ini dikenal sebagai

0/1 Knapsack Problem, yang merupakan salah satu masalah klasik dalam bidang ilmu komputer dan riset operasi.

Sebuah perusahaan ekspedisi sering dihadapkan pada situasi di mana mereka harus menentukan barang-barang mana yang akan dibawa dalam pengiriman dengan kapasitas kendaraan atau wadah yang terbatas. Setiap barang memiliki berat dan nilai keuntungan yang berbeda-beda, sehingga dibutuhkan algoritma yang efisien untuk menemukan kombinasi barang optimal.

Dua pendekatan algoritma yang umum digunakan untuk menyelesaikan masalah ini adalah

Branch and Bound dan Backtracking. Kedua algoritma ini termasuk dalam kategori pencarian pohon keputusan (decision tree search), namun memiliki strategi pruning yang berbeda. Laporan ini menyajikan implementasi, analisis, dan perbandingan kedua algoritma tersebut.

1.2 Rumusan Masalah

Rumusan masalah dalam laporan ini adalah:

1. Bagaimana cara kerja algoritma Branch and Bound dalam menyelesaikan 0/1 Knapsack Problem?
2. Bagaimana cara kerja algoritma Backtracking dalam menyelesaikan 0/1 Knapsack Problem?
3. Bagaimana perbandingan kompleksitas waktu dan ruang antara kedua algoritma tersebut?

BAB II

ANALISIS DAN LANGKAH PENYELESAIAN

2.1 Deskripsi Studi Kasus

Sebuah perusahaan ekspedisi memiliki kendaraan dengan kapasitas muatan maksimum sebesar 10 kg. Tersedia 4 jenis barang yang dapat dimuat, masing-masing dengan berat dan nilai keuntungan seperti pada tabel berikut:

Tabel 2.1 Data Barang

No	Nama Barang	Berat (kg)	Profit (nilai)	Rasio Profit/Berat
1	Barang A	2	40	20,00
2	Barang B	3	50	16,67
3	Barang C	5	100	20,00
4	Barang D	4	60	15,00

Tujuan: memilih kombinasi barang yang menghasilkan profit maksimum tanpa melebihi kapasitas 10 kg.

2.2 Konsep Dasar Branch and Bound

Branch and Bound adalah teknik pencarian yang membangun pohon keputusan secara sistematis. Setiap node pada pohon merepresentasikan keputusan untuk satu barang: diambil (include) atau tidak diambil (exclude).

Pada setiap node dihitung Upper Bound — estimasi profit maksimum yang masih bisa dicapai dari node tersebut. Jika Upper Bound $\leq$ solusi terbaik yang sudah ditemukan, maka cabang tersebut dipangkas (pruned) tanpa perlu ditelusuri lebih lanjut. Strategi ini secara drastis mengurangi jumlah node yang harus diperiksa.

Implementasi dalam laporan ini menggunakan Best-First Search melalui priority queue, di mana node dengan Upper Bound tertinggi diproses terlebih dahulu.

2.3 Rumus Bound

Upper Bound dihitung dengan pendekatan *fractional knapsack* (greedy):

$$UB = \text{profit_terkumpul} + \text{greedy_sisas_kapasitas}$$

Langkah perhitungan:

1. Urutkan barang berdasarkan rasio profit/berat secara menurun.
2. Ambil barang utuh selama kapasitas tersisa mencukupi.
3. Ambil sebagian barang berikutnya secara proporsional (fraksi) untuk kapasitas sisa.
4. Hasil penjumlahan adalah Upper Bound node tersebut.

Karena Backtracking tidak menggunakan Upper Bound, pruning hanya terjadi saat berat melebihi kapasitas (constraint violation).

BAB III LANGKAH PENYELESAIAN SECARA MANUAL

3.1 Contoh Data

Data yang digunakan (sudah diurutkan berdasarkan rasio profit/berat):

Barang	Berat	Profit	Rasio
A	2	40	20,00
C	5	100	20,00
B	3	50	16,67
D	4	60	15,00

3.2 Node Awal (Root)

Node awal (root) dimulai dengan kondisi kosong: profit = 0, berat = 0, level = 1.

Upper Bound root dihitung:

$$UB = 0 + 40 + 100 + 50 + (1/4) \times 60 = 205$$

3.3 Percabangan Barang A

Dari root, percabangkan Barang A:

- Node INCLUDE A: berat = 2, profit = 40, UB = 190
- Node EXCLUDE A: berat = 0, profit = 0, UB = 180

Node INCLUDE A (UB=190) lebih menjanjikan, diproses lebih dulu oleh Best-First Search.

3.4 Percabangan Setelah Mengambil A

Dari node INCLUDE A, percabangkan Barang C (urutan berikutnya):

- Node INCLUDE A,C: berat = 7, profit = 140, UB = 190
- Node EXCLUDE C dari A: berat = 2, profit = 40, UB = 150

3.5 Percabangan Setelah Mengambil A dan Tidak Mengambil B

Dari node INCLUDE A,C, percabangkan Barang B:

Node INCLUDE A,C,B: berat = 10, profit = 190, UB = 0 → berat tepat kapasitas, tidak bisa tambah D

Node EXCLUDE B dari A,C: berat = 7, profit = 140, UB = 185

Solusi sementara: A, C, B dengan profit = 190

Node EXCLUDE B memiliki UB = 185 < profit terbaik = 190, sehingga dipangkas

3.6 Percabangan Setelah Mengambil A dan C

Kembali ke node EXCLUDE A (UB=180), karena 180 < 190 → dipangkas

Kembali ke node EXCLUDE C dari A (UB=150), karena 150 < 190 → dipangkas

Semua node tersisa dipangkas karena Upper Bound-nya tidak melebihi profit terbaik (190).

3.7 Hasil Akhir Branch and Bound

Solusi optimal ditemukan: Barang A, B, C (urutan setelah sorting: A, C, B).

Keterangan	Nilai
Barang Terpilih	A, B, C
Total Berat	10 kg
Total Profit	190
Node Dikunjungi	5 node
Node Dipangkas	4 node

BAB IV

METODE BACKTRACKING

Backtracking adalah teknik pencarian sistematis dengan rekursi. Program mencoba setiap kemungkinan kombinasi barang secara mendalam (depth-first), dan membatalkan (backtrack) pilihan yang sudah terbukti tidak menghasilkan solusi valid.

4.1 Cara Kerja Backtracking

Untuk setiap barang pada level ke-i, terdapat dua pilihan rekursif:

AMBIL barang ke-i: tambahkan berat dan profit ke akumulator, rekursi ke i+1

SKIP barang ke-i: tidak mengubah akumulator, rekursi ke i+1

Pruning terjadi saat berat total melebihi kapasitas; cabang tersebut ditinggalkan. Setelah rekursi selesai (baik berhasil maupun dipangkas), program mundur dan mencoba pilihan lain.

4.2 Pohon Pencarian Backtracking

Dengan 4 barang (A, B, C, D) terdapat $2^4 = 16$ kemungkinan kombinasi. Backtracking menjelajahi 28 node karena setiap keputusan (AMBIL/SKIP) merupakan node tersendiri dalam pohon rekursi. Beberapa cabang dipangkas karena berat melampaui kapasitas 10 kg.

Keterangan	Nilai
Barang Terpilih	A, B, C
Total Berat	10 kg
Total Profit	190
Node Dikunjungi	28 node

4.3 Keunggulan dan Keterbatasan Backtracking

Keunggulan	Keterbatasan
Implementasi sederhana dan mudah dipahami.	Menjelajahi lebih banyak node dibandingkan Branch and Bound.
Penggunaan memori sangat efisien, yaitu $O(n)$ untuk stack rekursi.	Pruning hanya berdasarkan constraint (berat > kapasitas), bukan estimasi profit maksimum.
Tidak memerlukan struktur data tambahan seperti priority queue.	Kurang efisien untuk jumlah barang yang sangat besar.

BAB V

KOMPLEKSITAS ALGORITMA

5.1 Kompleksitas Backtracking

Kategori	Nilai	Keterangan
Kompleksitas Waktu (Worst Case)	$O(2^n)$	Menjelajahi seluruh kemungkinan kombinasi.
Kompleksitas Waktu (Average Case)	Eksponensial	Lebih baik karena pruning, tetapi tetap tumbuh secara eksponensial.
Kompleksitas Waktu ($n = 4$)	28 node	Mengunjungi 28 dari total maksimum 31 node.
Kompleksitas Ruang	$O(n)$	Menggunakan stack rekursi dengan kedalaman maksimum n .
Penggunaan Memori	Efisien	Tidak menyimpan semua node secara bersamaan.

5.2 Kompleksitas Branch and Bound

Kategori	Nilai	Keterangan
Kompleksitas Waktu (Worst Case)	$O(2^n)$	Jika proses pruning tidak efektif, algoritma dapat menjelajahi seluruh kemungkinan kombinasi seperti Backtracking.
Kompleksitas Waktu (Average Case)	Lebih baik dari $O(2^n)$	Pruning berdasarkan Upper Bound mampu mengurangi jumlah node yang perlu dieksplorasi sehingga pencarian menjadi lebih efisien.
Kompleksitas Waktu ($n = 4$)	5 node	Pada studi kasus, hanya 5 node yang dikunjungi untuk menemukan solusi optimal.

Kompleksitas Ruang (Worst Case)	$O(n \times 2^n)$	Priority queue dapat menyimpan banyak node aktif secara bersamaan selama proses pencarian.
Penggunaan Memori	Relatif besar	Membutuhkan memori lebih banyak dibandingkan Backtracking karena harus mempertahankan node-node aktif dalam priority queue.

5.3 Tabel Perbandingan

Tabel 5.1 Perbandingan Kedua Algoritma

Aspek	Branch and Bound	Backtracking
Strategi Dasar	Best-First Search + pruning bound	Depth-First Search + constraint
Pruning	Upper Bound (lebih agresif)	Violation constraint (kapasitas)
Waktu Worst Case	$O(2^n)$	$O(2^n)$
Waktu Average	Jauh lebih cepat	Cukup cepat
Ruang (Space)	$O(n \times 2^n)$ worst case	$O(n)$ — sangat hemat
Barang Terpilih	A, B, C	A, B, C
Total Berat	10 kg	10 kg
Total Profit	190	190
Node Dikunjungi	5 node	28 node
Node Dipangkas	4 node	Beberapa (berat)

Implementasi	Lebih kompleks	Lebih sederhana
Struktur Data	Priority Queue	Stack rekursi
Cocok Untuk	n besar, butuh kecepatan	n kecil, memori terbatas

BAB VI

PENJELASAN IMPLEMENTASI PROGRAM

6.1 Source Code

6.1.1 File branch_and_bound.cpp

File ini mengimplementasikan algoritma Branch and Bound murni menggunakan priority queue (Best-First Search). Barang diurutkan berdasarkan rasio profit/berat sebelum pencarian dimulai. Setiap node menyimpan level, profit, berat, upper bound, dan daftar barang yang dipilih.

Source Code branch_and_bound.cpp:

```
//
=====

// FILE: branch_and_bound.cpp
// JUDUL: Implementasi Branch and Bound untuk 0/1 Knapsack
Problem
// STUDI KASUS: Optimasi Pemilihan Barang pada Perusahaan
Ekspedisi
//
=====

#include <iostream>
#include <queue>
#include <vector>
#include <string>
#include <algorithm>
#include <chrono>
#include <iomanip>

using namespace std;

//
=====

// STRUKTUR DATA BARANG
// Menyimpan atribut setiap barang yang akan dipilih
//
=====

struct Barang {
```

```

    string nama;    // Nama barang
    int berat;      // Berat barang (kg)
    int profit;     // Keuntungan/profit barang (rupiah/unit)
    double rasio;   // Rasio profit/berat (digunakan untuk
upper bound)
};

//
=====

// STRUKTUR DATA NODE (Simpul Pohon Pencarian)
// Setiap node merepresentasikan keputusan untuk 1 barang
//
=====

struct Node {
    int level;          // Level/kedalaman dalam pohon (indeks
barang ke-)
    int profit;         // Total profit yang sudah dikumpulkan
    int berat;          // Total berat yang sudah dikumpulkan
    double upperBound;  // Estimasi profit maksimum yang bisa
dicapai
    vector<int> pilihan; // Daftar indeks barang yang dipilih
di node ini
};

//
=====

// VARIABEL GLOBAL
//
=====

int jumlahBarang;
int kapasitas;
vector<Barang> daftarBarang;

int nodesDikunjungi = 0;    // Counter node yang dikunjungi
int nodesDipangkas = 0;    // Counter node yang dipangkas
(pruned)

//
=====

// FUNGSI: Hitung Upper Bound (Batas Atas)
// Menggunakan pendekatan fractional knapsack (greedy) untuk

```



```

// mengestimasi profit maksimum yang bisa diraih dari node
ini.
// Jika upper bound <= profit terbaik saat ini, node
dipangkas.
//
=====

double hitungUpperBound(Node& node) {
    // Jika berat sudah melebihi kapasitas, tidak ada solusi
    valid
    if (node.berat >= kapasitas) return 0;

    double bound = node.profit;
    int beratSisa = kapasitas - node.berat;
    int i = node.level + 1;

    // Ambil barang secara greedy berdasarkan rasio
    profit/berat
    while (i < jumlahBarang && beratSisa >=
    daftarBarang[i].berat) {
        beratSisa -= daftarBarang[i].berat;
        bound      += daftarBarang[i].profit;
        i++;
    }

    // Jika masih ada kapasitas tersisa, ambil sebagian barang
    berikutnya
    // (ini adalah estimasi - tidak boleh dilakukan di solusi
    nyata 0/1)
    if (i < jumlahBarang) {
        bound += (double)beratSisa * daftarBarang[i].rasio;
    }

    return bound;
}

//
=====
// FUNGSI: Algoritma Branch and Bound
// Menggunakan priority queue (Best-First Search) untuk
menjelajahi
// node dengan upper bound tertinggi terlebih dahulu.

```

```

//
=====
void branchAndBound() {
    // Urutkan barang berdasarkan rasio profit/berat
    (descending)
    // Ini penting agar upper bound lebih akurat
    sort(daftarBarang.begin(), daftarBarang.end(),
        [](const Barang& a, const Barang& b) {
            return a.rasio > b.rasio;
        });

    cout << "\n[INFO] Urutan barang setelah diurutkan
    berdasarkan rasio profit/berat:\n";
    cout << left << setw(12) << "Nama"
        << setw(8) << "Berat"
        << setw(10) << "Profit"
        << setw(10) << "Rasio" << "\n";
    cout << string(40, '-') << "\n";
    for (auto& b : daftarBarang) {
        cout << left << setw(12) << b.nama
            << setw(8) << b.berat
            << setw(10) << b.profit
            << fixed << setprecision(2) << setw(10) <<
b.rasio << "\n";
    }

    // Priority queue: node dengan upper bound tertinggi
    diproses duluan
    // (Best-First Search)
    auto cmp = [](const Node& a, const Node& b) {
        return a.upperBound < b.upperBound;
    };
    priority_queue<Node, vector<Node>, decltype(cmp)> pq(cmp);

    // Inisialisasi node akar (root)
    Node root;
    root.level      = -1;    // Belum memilih barang apapun
    root.profit      = 0;
    root.berat       = 0;
    root.upperBound = hitungUpperBound(root);
    root.pilihan     = {};
    pq.push(root);
}

```

```

int profitTerbaik = 0;
vector<int> pilihanTerbaik;

cout << "\n[PROSES] Penelusuran Node:\n";
cout << string(70, '=') << "\n";
cout << left << setw(8) << "Node"
    << setw(8) << "Level"
    << setw(10) << "Profit"
    << setw(8) << "Berat"
    << setw(14) << "UpperBound"
    << "Status" << "\n";
cout << string(70, '-') << "\n";

int nodeId = 0;

//
=====

// LOOP UTAMA: Proses setiap node dari priority queue
//
=====

while (!pq.empty()) {
    Node curr = pq.top();
    pq.pop();
    nodesDikunjungi++;
    nodeId++;

    // Cek apakah node ini masih layak dijelajahi
    // (upper bound harus lebih besar dari solusi terbaik
    saat ini)
    if (curr.upperBound <= profitTerbaik) {
        nodesDipangkas++;
        cout << left << setw(8) << nodeId
            << setw(8) << curr.level + 1
            << setw(10) << curr.profit
            << setw(8) << curr.berat
            << setw(14) << fixed << setprecision(2) <<
curr.upperBound
            << "DIPANGKAS (bound <= best)\n";
        continue;
    }
}

```

```

        // Jika sudah mencapai daun pohon (semua barang sudah
        dipertimbangkan)
        if (curr.level == jumlahBarang - 1) {
            cout << left << setw(8) << nodeId
                << setw(8) << curr.level + 1
                << setw(10) << curr.profit
                << setw(8) << curr.berat
                << setw(14) << fixed << setprecision(2) <<
curr.upperBound
                << "DAUN (leaf node)\n";
            continue;
        }

        // Pindah ke barang berikutnya
        int nextLevel = curr.level + 1;

        // -----
        // CABANG KIRI: Masukkan barang ke dalam tas (include)
        // -----
        -----
        Node include;
        include.level = nextLevel;
        include.berat = curr.berat +
daftarBarang[nextLevel].berat;
        include.profit = curr.profit +
daftarBarang[nextLevel].profit;
        include.pilihan = curr.pilihan;
        include.pilihan.push_back(nextLevel);

        // Hanya masukkan jika berat tidak melebihi kapasitas
        if (include.berat <= kapasitas) {
            include.upperBound = hitungUpperBound(include);

            // Perbarui solusi terbaik jika profit lebih
tinggi
            if (include.profit > profitTerbaik) {
                profitTerbaik = include.profit;
                pilihanTerbaik = include.pilihan;
            }
        }
    }
}

```

```

// Tambahkan ke antrian hanya jika masih
menjanjikan
    if (include.upperBound > profitTerbaik) {
        pq.push(include);
        cout << left << setw(8) << ++nodeId
            << setw(8) << include.level + 1
            << setw(10) << include.profit
            << setw(8) << include.berat
            << setw(14) << fixed << setprecision(2)
        << include.upperBound
            << "INCLUDE " <<
        daftarBarang[nextLevel].nama << "\n";
    } else {
        nodesDipangkas++;
        cout << left << setw(8) << ++nodeId
            << setw(8) << include.level + 1
            << setw(10) << include.profit
            << setw(8) << include.berat
            << setw(14) << fixed << setprecision(2)
        << include.upperBound
            << "DIPANGKAS (include " <<
        daftarBarang[nextLevel].nama << ") \n";
    }
} else {
    nodesDipangkas++;
    cout << left << setw(8) << ++nodeId
        << setw(8) << nextLevel + 1
        << setw(10) << include.profit
        << setw(8) << include.berat
        << setw(14) << setw(14) << "N/A"
        << "DIPANGKAS (berat melebihi kapasitas) \n";
}

// -----
-----
// CABANG KANAN: Tidak masukkan barang (exclude)
// -----
-----

Node exclude;
exclude.level      = nextLevel;
exclude.berat      = curr.berat;
exclude.profit     = curr.profit;

```

```

        exclude.pilihan      = curr.pilihan;
        exclude.upperBound = hitungUpperBound(exclude);

        if (exclude.upperBound > profitTerbaik) {
            pq.push(exclude);
            cout << left << setw(8)  << ++nodeId
                  << setw(8)  << exclude.level + 1
                  << setw(10) << exclude.profit
                  << setw(8)  << exclude.berat
                  << setw(14) << fixed << setprecision(2) <<
exclude.upperBound
                  << "EXCLUDE " << daftarBarang[nextLevel].nama
<< "\n";
        } else {
            nodesDipangkas++;
            cout << left << setw(8)  << ++nodeId
                  << setw(8)  << exclude.level + 1
                  << setw(10) << exclude.profit
                  << setw(8)  << exclude.berat
                  << setw(14) << fixed << setprecision(2) <<
exclude.upperBound
                  << "DIPANGKAS (exclude " <<
daftarBarang[nextLevel].nama << ") \n";
        }
    }

    cout << string(70, '=') << "\n";

    //
=====
    // TAMPILKAN HASIL AKHIR
    //
=====

    int totalBerat  = 0;
    int totalProfit = 0;
    string namaBarangDipilih = "";

    for (int idx : pilihanTerbaik) {
        totalBerat += daftarBarang[idx].berat;
        totalProfit += daftarBarang[idx].profit;
        if (!namaBarangDipilih.empty()) namaBarangDipilih += "
";
    }

```

```

        namaBarangDipilih += daftarBarang[idx].nama;
    }

    cout << "\n";
    cout << string(40, '=') << "\n";
    cout << "            HASIL OPTIMASI (Branch and Bound)\n";
    cout << string(40, '=') << "\n";
    cout << "Barang Terpilih : " << namaBarangDipilih <<
"\n";
    cout << "Total Berat      : " << totalBerat << " kg\n";
    cout << "Total Profit      : " << totalProfit << "\n";
    cout << "Node Dikunjungi : " << nodesDikunjungi << "\n";
    cout << "Node Dipangkas   : " << nodesDipangkas << "\n";
}

//
=====
// FUNGSI UTAMA (main)
//
=====
int main() {
    cout <<
"=====\\
n";
    cout << "    BRANCH AND BOUND - 0/1 KNAPSACK PROBLEM\n";
    cout << "    Studi Kasus: Optimasi Pemilihan Barang
Ekspedisi\n";
    cout <<
"=====\\
n";

    //
=====
    // INISIALISASI DATA BARANG
    // Sesuai soal: 4 barang dengan berat dan profit yang
ditetapkan
    //
=====
    daftarBarang = {
        {"A", 2, 40, (double)40 / 2}, // Barang A: berat 2,
profit 40

```

```

        {"B", 3, 50, (double)50 / 3}, // Barang B: berat 3,
profit 50
        {"C", 5, 100, (double)100 / 5}, // Barang C: berat 5,
profit 100
        {"D", 4, 60, (double)60 / 4} // Barang D: berat 4,
profit 60
    };
    jumlahBarang = daftarBarang.size();
    kapasitas = 10;

    // Tampilkan data input
    cout << "\n[DATA INPUT]\n";
    cout << "Kapasitas Knapsack : " << kapasitas << " kg\n";
    cout << "Jumlah Barang : " << jumlahBarang << "\n\n";
    cout << left << setw(12) << "Nama"
        << setw(8) << "Berat"
        << setw(10) << "Profit"
        << setw(10) << "Rasio" << "\n";
    cout << string(40, '-') << "\n";
    for (auto& b : daftarBarang) {
        cout << left << setw(12) << b.nama
            << setw(8) << b.berat
            << setw(10) << b.profit
            << fixed << setprecision(2) << setw(10) <<
b.rasio << "\n";
    }

    //
=====

    // JALANKAN ALGORITMA DAN UKUR WAKTU EKSEKUSI
    // Menggunakan std::chrono untuk presisi tinggi
(microseconds)
    //
=====

    auto mulai = chrono::high_resolution_clock::now();
    branchAndBound();
    auto selesai = chrono::high_resolution_clock::now();

    auto durasi =
chrono::duration_cast<chrono::microseconds>(selesai -
mulai).count();

```



```

        cout << "Waktu Eksekusi   : " << durasi << "
microseconds\n";
        cout << string(40, '=') << "\n";

        return 0;
    }

```

6.1.2 File backtracking.cpp

File ini mengimplementasikan algoritma Backtracking rekursif. Setiap pemanggilan fungsi backtrack() mencoba dua pilihan (AMBIL/SKIP) untuk setiap barang. Proses pencarian divisualisasikan dengan indentasi yang menunjukkan kedalaman rekursi.

Source Code backtracking.cpp:

```

//
=====
// FILE: backtracking.cpp
// JUDUL: Implementasi Backtracking untuk 0/1 Knapsack Problem
// STUDI KASUS: Optimasi Pemilihan Barang pada Perusahaan
Ekspedisi
//
=====

#include <iostream>
#include <vector>
#include <string>
#include <chrono>
#include <iomanip>

using namespace std;

//
=====
// STRUKTUR DATA BARANG
// Menyimpan atribut setiap barang yang akan dipilih

```

```

//
=====

struct Barang {
    string nama;    // Nama barang
    int berat;      // Berat barang (kg)
    int profit;     // Keuntungan/profit barang
};

//
=====
// VARIABEL GLOBAL
//
=====

int jumlahBarang;
int kapasitas;
vector<Barang> daftarBarang;

int profitTerbaik = 0;    // Menyimpan profit solusi terbaik
yang ditemukan
int nodesDikunjungi = 0; // Counter jumlah node yang
dikunjungi

vector<int> pilihanSaatIni; // Barang yang sedang
dipertimbangkan
vector<int> pilihanTerbaik; // Barang pada solusi terbaik

//
=====
// FUNGSI: Tampilkan Indentasi (visualisasi kedalaman rekursi)
// Membantu mahasiswa memahami kedalaman pohon pencarian
//
=====
void tampilkanIndentasi(int level) {
    for (int i = 0; i < level; i++) cout << " | ";
}

//
=====
// FUNGSI REKURSIF: Backtracking
//
// Parameter:
//   - level      : Indeks barang yang sedang dipertimbangkan

```

```

// - beratSaatIni: Total berat barang yang sudah dipilih
// - profitSaatIni: Total profit barang yang sudah dipilih
//
// Cara Kerja:
// 1. Untuk setiap barang, coba DUA pilihan: AMBIL atau
TIDAK AMBIL
// 2. Jika mengambil barang menyebabkan berat melebihi
kapasitas → BATALKAN
// 3. Jika sudah mencapai semua barang → catat jika solusi
ini lebih baik
// 4. Setelah menjelajahi satu cabang → BACKTRACK (kembali
ke kondisi sebelumnya)
//
=====
void backtrack(int level, int beratSaatIni, int profitSaatIni)
{
    // -----
    -
    // BASE CASE: Sudah mempertimbangkan semua barang
    // -----
    -

    if (level == jumlahBarang) {
        nodesDikunjungi++;

        // Perbarui solusi terbaik jika ditemukan profit yang
lebih besar
        if (profitSaatIni > profitTerbaik) {
            profitTerbaik = profitSaatIni;
            pilihanTerbaik = pilihanSaatIni;

            cout << string(50, '-') << "\n";
            cout << "[SOLUSI BARU DITEMUKAN!] Profit = " <<
profitTerbaik << "\n";
            cout << "Barang dipilih: ";
            for (int idx : pilihanTerbaik) cout <<
daftarBarang[idx].nama << " ";
            cout << "\n" << string(50, '-') << "\n";
        } else {
            tampilkanIndentasi(level);
            cout << "-> Leaf: profit=" << profitSaatIni
<< " (tidak lebih baik dari " <<
profitTerbaik << ")\n";
        }
    }
}

```

```

        return;
    }

    nodesDikunjungi++;

    // -----
-
    // CABANG KIRI: AMBIL barang ke-level
    // -----
-

    int beratBaru = beratSaatIni +
daftarBarang[level].berat;
    int profitBaru = profitSaatIni +
daftarBarang[level].profit;

    tampilkanIndentasi(level);
    cout << "+-[AMBIL] Barang " << daftarBarang[level].nama
        << " (berat=" << daftarBarang[level].berat
        << ", profit=" << daftarBarang[level].profit << ")"
        << " | Total berat=" << beratBaru
        << ", Total profit=" << profitBaru << "\n";

    // Periksa apakah berat masih dalam kapasitas
    if (beratBaru <= kapasitas) {
        // Masukkan barang ke dalam pilihan saat ini
        pilihanSaatIni.push_back(level);

        // Rekursi ke barang berikutnya (telusuri lebih dalam)
        backtrack(level + 1, beratBaru, profitBaru);

        // BACKTRACK: Keluarkan barang dari pilihan (batalkan
        keputusan)
        pilihanSaatIni.pop_back();

        tampilkanIndentasi(level);
        cout << " [BACKTRACK] Kembali dari Barang " <<
daftarBarang[level].nama << "\n";
    } else {
        // Berat melebihi kapasitas → cabang ini dipangkas
        tampilkanIndentasi(level);
        cout << " [BATALKAN] Berat " << beratBaru

```

```

        << " melebihi kapasitas " << kapasitas << " →
Cabang dipangkas!\n";
    }

    // -----
-
    // CABANG KANAN: TIDAK AMBIL barang ke-level
    // -----
-

    tampilkanIndentasi(level);
    cout << "+-[SKIP]  Barang " << daftarBarang[level].nama
        << " dilewati"
        << " | Total berat=" << beratSaatIni
        << ", Total profit=" << profitSaatIni << "\n";

    // Rekursi ke barang berikutnya tanpa mengambil barang ini
    backtrack(level + 1, beratSaatIni, profitSaatIni);

    tampilkanIndentasi(level);
    cout << "  [BACKTRACK] Kembali setelah skip Barang " <<
daftarBarang[level].nama << "\n";
}

//
=====

// FUNGSI UTAMA (main)
//
=====

int main() {
    cout <<
    "=====\\
n";
    cout << "    BACKTRACKING - 0/1 KNAPSACK PROBLEM\n";
    cout << "    Studi Kasus: Optimasi Pemilihan Barang
Ekspedisi\n";
    cout <<
    "=====\\
n";

    //
=====

    // INISIALISASI DATA BARANG

```

```

// Sesuai soal: 4 barang dengan berat dan profit yang
ditentukan
//
=====

daftarBarang = {
    {"A", 2, 40},    // Barang A: berat 2, profit 40
    {"B", 3, 50},    // Barang B: berat 3, profit 50
    {"C", 5, 100},   // Barang C: berat 5, profit 100
    {"D", 4, 60}     // Barang D: berat 4, profit 60
};
jumlahBarang = daftarBarang.size();
kapasitas    = 10;

// Tampilkan data input
cout << "\n[DATA INPUT]\n";
cout << "Kapasitas Knapsack : " << kapasitas << " kg\n";
cout << "Jumlah Barang      : " << jumlahBarang << "\n\n";
cout << left << setw(12) << "Nama"
    << setw(8)  << "Berat"
    << setw(10) << "Profit" << "\n";
cout << string(30, '-') << "\n";
for (auto& b : daftarBarang) {
    cout << left << setw(12) << b.nama
        << setw(8)  << b.berat
        << setw(10) << b.profit << "\n";
}

//
=====

// JALANKAN BACKTRACKING DAN UKUR WAKTU EKSEKUSI
//
=====

cout << "\n[PROSES] Penelusuran Backtracking:\n";
cout << string(60, '=') << "\n";
cout << "Keterangan:\n";
cout << "  [AMBIL]      = Barang dimasukkan ke dalam
tas\n";
cout << "  [SKIP]       = Barang tidak diambil\n";
cout << "  [BATALKAN]    = Cabang dipangkas karena berat
melebihi kapasitas\n";
cout << "  [BACKTRACK] = Kembali ke keputusan
sebelumnya\n";
cout << string(60, '=') << "\n\n";

```

```

        auto mulai = chrono::high_resolution_clock::now();
        backtrack(0, 0, 0);
        auto selesai = chrono::high_resolution_clock::now();

        auto durasi =
        chrono::duration_cast<chrono::microseconds>(selesai -
        mulai).count();

        //
        =====
        // TAMPILKAN HASIL AKHIR
        //
        =====

        int totalBerat = 0;
        int totalProfit = 0;
        string namaBarangDipilih = "";

        for (int idx : pilihanTerbaik) {
            totalBerat += daftarBarang[idx].berat;
            totalProfit += daftarBarang[idx].profit;
            if (!namaBarangDipilih.empty()) namaBarangDipilih += "
";
            namaBarangDipilih += daftarBarang[idx].nama;
        }

        cout << "\n";
        cout << string(40, '=') << "\n";
        cout << "          HASIL OPTIMASI (Backtracking)\n";
        cout << string(40, '=') << "\n";
        cout << "Barang Terpilih : " << namaBarangDipilih <<
"\n";
        cout << "Total Berat      : " << totalBerat << " kg\n";
        cout << "Total Profit      : " << totalProfit << "\n";
        cout << "Node Dikunjungi : " << nodesDikunjungi << "\n";
        cout << "Waktu Eksekusi   : " << durasi << "
microseconds\n";
        cout << string(40, '=') << "\n";

        return 0;
    }

```

--

6.2 Struktur Data yang Digunakan

Struktur Data	Algoritma	Kegunaan
struct Barang	Keduanya	Menyimpan nama, berat, profit barang
struct Node	Branch & Bound	Merepresentasikan simpul pohon pencarian
priority_queue	Branch & Bound	Best-First Search, node UB tertinggi duluan
vector<int>	Keduanya	Menyimpan daftar barang terpilih
Stack rekursi	Backtracking	Menyimpan state rekursi secara implisit
chrono	Keduanya	Mengukur waktu eksekusi (microseconds)

6.3 Implementasi Branch and Bound

Fungsi hitungUpperBound(Node& node):

Menghitung estimasi profit maksimum dari node tertentu menggunakan greedy fractional. Barang diambil utuh selama kapasitas mencukupi; sisanya diambil secara proporsional.

Fungsi branchAndBound():

Menggunakan priority_queue dengan custom comparator (max-heap berdasarkan upperBound). Setiap iterasi mengambil node dengan upperBound tertinggi, lalu membuat dua child node (include dan exclude). Node yang tidak menjanjikan ($UB \leq \text{profitTerbaik}$) dipangkas.

6.4 Implementasi Backtracking

Fungsi backtrack(level, beratSaatIni, profitSaatIni):

Fungsi rekursif yang mencoba AMBIL dan SKIP untuk setiap barang. Saat base case tercapai (level == jumlahBarang), solusi dibandingkan dengan profitTerbaik. Setelah rekursi, state dikembalikan ke kondisi sebelumnya (pop dari vector pilihan).

BAB VII

KESIMPULAN

Berdasarkan implementasi dan analisis yang telah dilakukan, dapat disimpulkan bahwa kedua algoritma, yaitu Branch and Bound dan Backtracking, berhasil menghasilkan solusi optimal yang sama pada kasus 0/1 Knapsack Problem. Solusi terbaik diperoleh dengan memilih Barang A, Barang B, dan Barang C sehingga total berat yang digunakan adalah 10 kg dengan total keuntungan sebesar 190.

Dari sisi efisiensi pencarian, algoritma Branch and Bound menunjukkan performa yang lebih baik dibandingkan Backtracking. Pada data yang digunakan, Branch and Bound hanya perlu mengunjungi 5 node untuk menemukan solusi optimal, sedangkan Backtracking harus mengunjungi 28 node. Hal ini menunjukkan bahwa Branch and Bound mampu mengurangi jumlah pencarian secara signifikan melalui proses pemangkasan (pruning) yang lebih efektif.

Perbedaan utama kedua algoritma terletak pada mekanisme pruning yang digunakan. Branch and Bound memanfaatkan nilai Upper Bound yang dihitung menggunakan konsep fractional knapsack untuk memperkirakan keuntungan maksimum yang mungkin diperoleh dari suatu cabang. Dengan pendekatan ini, cabang yang tidak berpotensi menghasilkan solusi lebih baik dapat dipangkas lebih awal. Sementara itu, Backtracking hanya melakukan pemangkasan ketika total berat barang yang dipilih telah melebihi kapasitas knapsack, sehingga jumlah cabang yang dieksplorasi cenderung lebih banyak.

Dari aspek penggunaan memori, Backtracking memiliki keunggulan karena hanya membutuhkan stack rekursi dengan kompleksitas ruang sebesar $O(n)$. Sebaliknya, Branch and Bound memerlukan struktur data tambahan berupa priority queue untuk menyimpan node-node yang akan dieksplorasi, sehingga kebutuhan memorinya lebih besar dan pada kasus terburuk dapat mencapai $O(n \cdot 2^n)$.

Secara keseluruhan, Branch and Bound lebih direkomendasikan untuk permasalahan dengan jumlah data yang besar karena mampu menemukan solusi optimal dengan waktu pencarian yang lebih cepat. Namun, untuk kasus dengan jumlah data yang relatif kecil atau ketika keterbatasan memori menjadi pertimbangan utama, Backtracking tetap menjadi pilihan yang baik karena implementasinya lebih sederhana dan mudah dipahami.

KONTRIBUSI ANGGOTA KELOMPOK

No	Nama	NIM	Kontribusi
1	Muhammad Dimas Setiaji	123240240	Implementasi program Branch and Bound (branch_and_bound.cpp), penyusunan Bab II dan Bab III.
2	Bintang Shada Kawibya Putra	123240247	Implementasi program Backtracking (backtracking.cpp), penyusunan Bab I dan Bab IV
3	Ajrun Ramadhan	123240254	Analisis kompleksitas algoritma, perbandingan kinerja Branch and Bound dan Backtracking, penyusunan Bab V serta Bab VI.
4	Noven Bugijangge	123240264	Penyusunan dan penggabungan laporan akhir, pembuatan cover, daftar isi, format dokumen, serta penyusunan Bab VII .